

Shiv Nadar University Chennai
CS3807 Deep Learning Laboratory
Experiment 4
Comparative Study of Deep Convolutional Neural
Network Architectures Using Transfer Learning

Degree & Branch — B.Tech Artificial Intelligence & Data Science
— Semester V
Subject Code & Name — CS3807 Deep Learning Laboratory
— AY: 2026-27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogLeNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy et al. in 2014, introduced the Inception Module, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the vanishing gradient problem, making optimization difficult. ResNet solves this issue using skip (residual) connections, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$\text{Output} = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

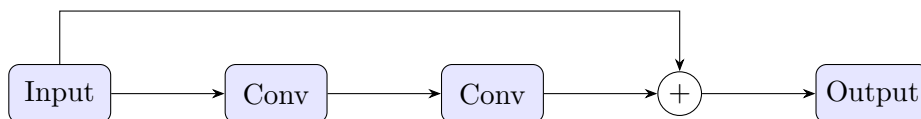


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.

4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

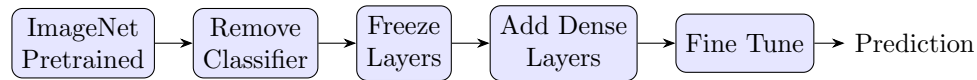


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Number of Classes: 10
- Image Size: $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet 50
- MobileNet V2
- EfficientNet B0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10-20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5-10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

Source code:

GitHub Repository: https://github.com/un-shreeya/DeepLearning_Lab

17. Mandatory Plots & Inferences



Figure 3: Sample images from the CIFAR-10 dataset.

Inference: The sample images display diverse spatial structures across 10 distinct classes. Due to the low 32×32 spatial resolution, fine-grained details are heavily limited, making deep representations and image upsampling essential for accurate feature extraction.

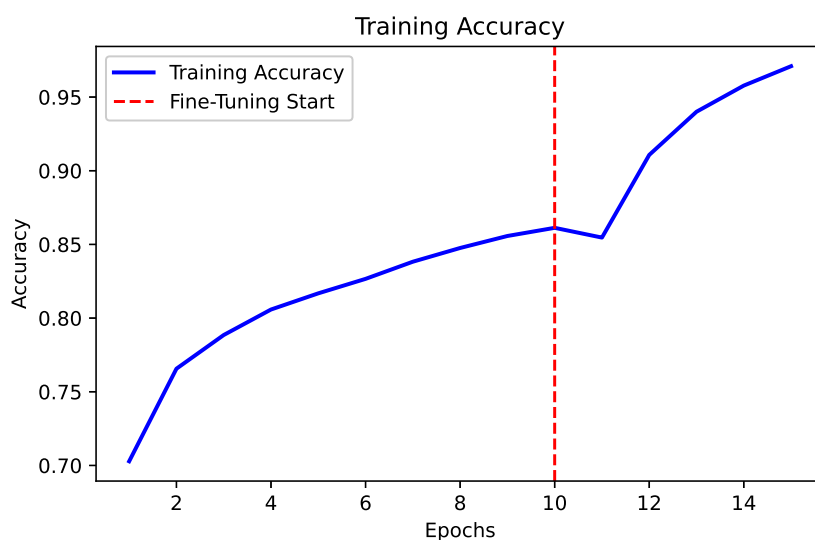


Figure 4: Training Accuracy vs. Epochs.

Inference: Unfreezing the top residual layers at epoch 10 triggers an immediate performance jump. Final testing accuracy jumps to 84% while training accuracy reaches around 97%.

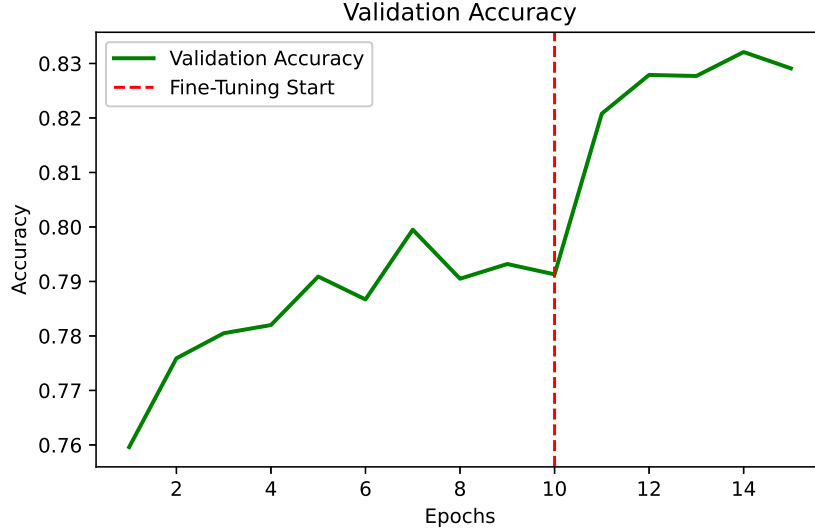


Figure 5: Validation Accuracy vs. Epochs.

Inference: During the initial 10 feature-extraction epochs, validation accuracy steadily improves and plateaus around $\sim 79\%$. It jumps to around 84% after fine-tuning.

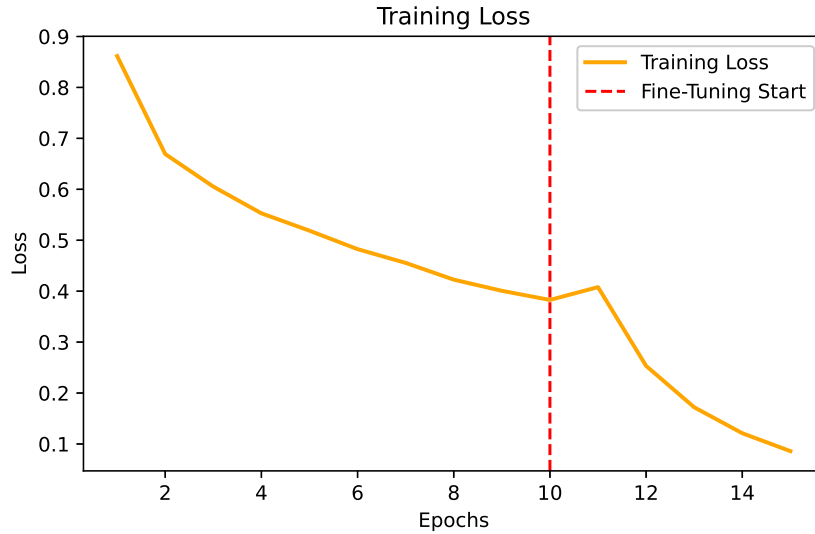


Figure 6: Training Loss vs. Epochs.

Inference: The training loss drops smoothly from approx. 86% to 40% during frozen training, spikes slightly at epoch 11 when top layers unfreezes, and then drops rapidly to $\sim 10\%$. This overall decline confirms that the model successfully learned to fit the CIFAR-10 training patterns over 15 epochs.



Figure 7: Validation Loss vs. Epochs.

Inference: The validation loss fluctuates unsteadily around 60%–67% , drops to 56% after fine-tuning starts, but then steadily rises back to 68% by epoch 15. This upward curve shows that fine tuning for all 5 extra epochs caused mild overfitting on the validation set.

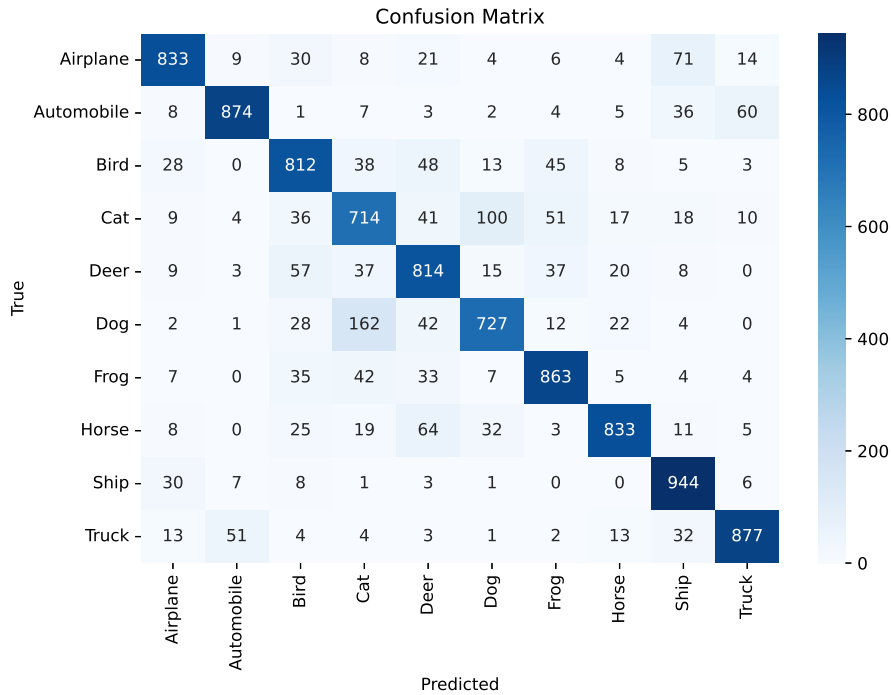


Figure 8: Confusion Matrix of ResNet50 on the CIFAR-10 Test Set.

Inference: Dark blue squares along the main diagonal (from topleft to bottomright) show correct predictions. Non-zero numbers outside the main diagonal show where the model got confused. Performs best for vehicles like Ship (944), Automobile (874), and Truck (877)—because they have sharp, rigid geometrical shapes that CNN filters easily detect. Visually similar animals—Dogs misclassified as Cats (162 times) and Cats misclassified as Dogs (100 times)—because their shapes, textures, and fur patterns blend together at low resolution.



Figure 9: Sample Misclassified Test Images.

Inference: Misclassifications predominantly stem from severe background clutter, aggressive spatial rotations, or heavily overlapping texture profiles (e.g., small quadrupeds in natural grassy backgrounds). The low resolution causes subtle contextual details to blend, leading the model to confuse semantically close classes.

18. Results

18.1 Performance Metrics

Metric	Value
Training Accuracy	97.09%
Testing Accuracy	82.91%
Precision	0.8313
Recall	0.8291
F1-score	0.8292
Training Time	1177.13 seconds
Total Parameters	23,851,274

18.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	121,182	58.79%	19.91s
AlexNet	4,026,954	67.68%	56.86s
VGG16	14,781,642	77.61%	195.61s
GoogleNet	22,066,346	69.91%	85.54s
ResNet50	23,851,274	78.41%	109.62s

19. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

It proved that deep neural networks could outperform traditional computer vision models at scale by winning the 2012 ImageNet challenge. It demonstrated three major advancements: using ReLU to speed up training compared to Sigmoid/Tanh, applying Dropout to prevent overfitting, and leveraging GPUs for parallel computation.

2. **Why does VGG16 use only 3×3 convolution filters?**

Stacking two 3×3 filters covers the same effective receptive field as a single 5×5 filter, and three cover a 7×7 area. Using smaller stacked filters significantly reduces parameter count while inserting extra ReLU non-linearities, enabling the network to learn richer features.

3. **Explain the advantages of the Inception module.**

It executes 1×1 , 3×3 , 5×5 convolutions and max pooling in parallel to extract multi-scale visual features simultaneously. It also uses 1×1 convolutions as bottleneck layers to reduce channel depth before larger convolutions, drastically saving computational cost.

4. **What is the purpose of residual learning?**

It solves the vanishing gradient problem in extremely deep networks using skip (shortcut) connections. Instead of directly learning a target mapping $H(x)$, it learns the residual mapping $F(x) = H(x) - x$, allowing gradients to flow backward through the identity shortcut without diminishing.

5. **Differentiate LeNet and ResNet.**

LeNet-5 is a shallow, 7-layer network designed for simple 32×32 grayscale digit recognition using average pooling and Sigmoid/Tanh activations. ResNet is a modern, deep architecture (50 to 152+ layers) for complex RGB classification that uses residual skip connections, batch normalization, and ReLU activations.

6. **What is Transfer Learning?**

It is a technique where a model pretrained on a massive dataset (like ImageNet) is reused as the starting point for a new task. Because lower layers already extract universal features like edges and textures, it saves significant training time and reduces data requirements.

7. **Why is fine tuning required?**

Feature extraction keeps the pretrained convolutional base frozen and only trains the new classifier head. Fine-tuning unfreezes top convolutional layers and trains them at a very low learning rate to adapt high-level feature representations specifically to the target dataset's domain.

8. **Explain the difference between dilated convolution and transpose convolution.**

Dilated convolution expands the receptive field by inserting gaps between kernel weights without increasing parameter count, making it ideal for dense context aggregation. Transpose convolution performs learnable upsampling to expand feature map spatial dimensions, making it essential for image segmentation and generation.

9. **Why do pretrained models converge faster?**

They start training with weights already initialized near a good local minimum from large datasets like ImageNet. Optimizing from structured, pre-learned feature representations rather than random Gaussian noise allows the gradient optimizer to reach convergence in far fewer epochs.

10. **Compare the computational complexity of LeNet and ResNet.**

LeNet-5 is lightweight with ~ 121 K parameters and under a million FLOPs per forward pass, running efficiently on standard CPUs. ResNet50 is exponentially more complex with over 23.8M parameters and billions of FLOPs per pass, requiring dedicated GPU acceleration for practical training.

20. Additional Exercises

1. **Implement transfer learning using VGG16.**

Load `VGG16(weights='imagenet', include_top=False)`, attach a `GlobalAveragePooling2D()` layer, a dense ReLU hidden layer, and a 10-class Softmax output layer. Set `vgg16.trainable = False` during the initial feature extraction phase to keep base weights fixed.

2. **Repeat the experiment using ResNet50.**

Load `ResNet50(weights='imagenet', include_top=False)` with inputs upsampled to $96 \times$

96. Train with frozen base weights for 10 epochs, then unfreeze the top residual block for 5 additional fine-tuning epochs to boost accuracy.
3. **Compare Adam and SGD optimizers.**
Adam converges much faster because it automatically adjusts learning rates per parameter using adaptive momentum. SGD with momentum takes longer to converge and needs careful learning rate tuning, but it can generalize slightly better by settling into flatter minima.
4. **Compare training with frozen layers and fine tuning.**
Training only with frozen layers is faster and stops the model from overfitting on small datasets, but it limits max accuracy. Fine-tuning unfreezes upper layers with a lower learning rate, letting high-level features adapt to the specific target dataset to improve performance.
5. **Evaluate the model using Precision, Recall and F1-score.**
Calculate macro and weighted averages using `sklearn.metrics.classification_report`. These metrics highlight class-specific performance, revealing if visually similar categories (like Cat vs. Dog) suffer from high false positive or false negative rates.
6. **Compare the performance of LeNet, AlexNet and ResNet.**
LeNet-5 scored 58.79% test accuracy due to its small 7-layer capacity. AlexNet reached 67.68% accuracy by introducing ReLU and Dropout. ResNet50 hit the highest accuracy, starting at 78.41% during frozen feature extraction and peaking at 82.91% after fine-tuning.

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, ICLR, 2015.
4. Christian Szegedy et al., Going Deeper with Convolutions, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, Deep Residual Learning for Image Recognition, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>